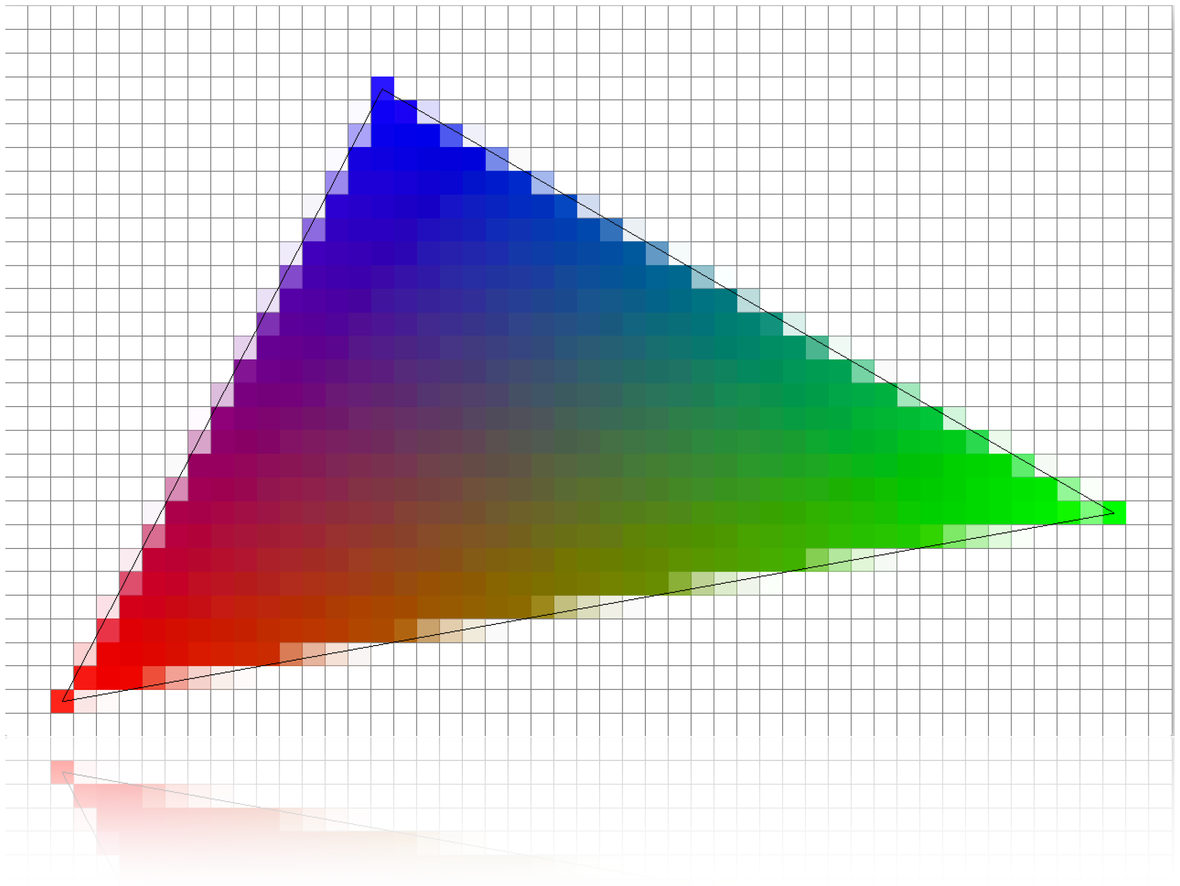
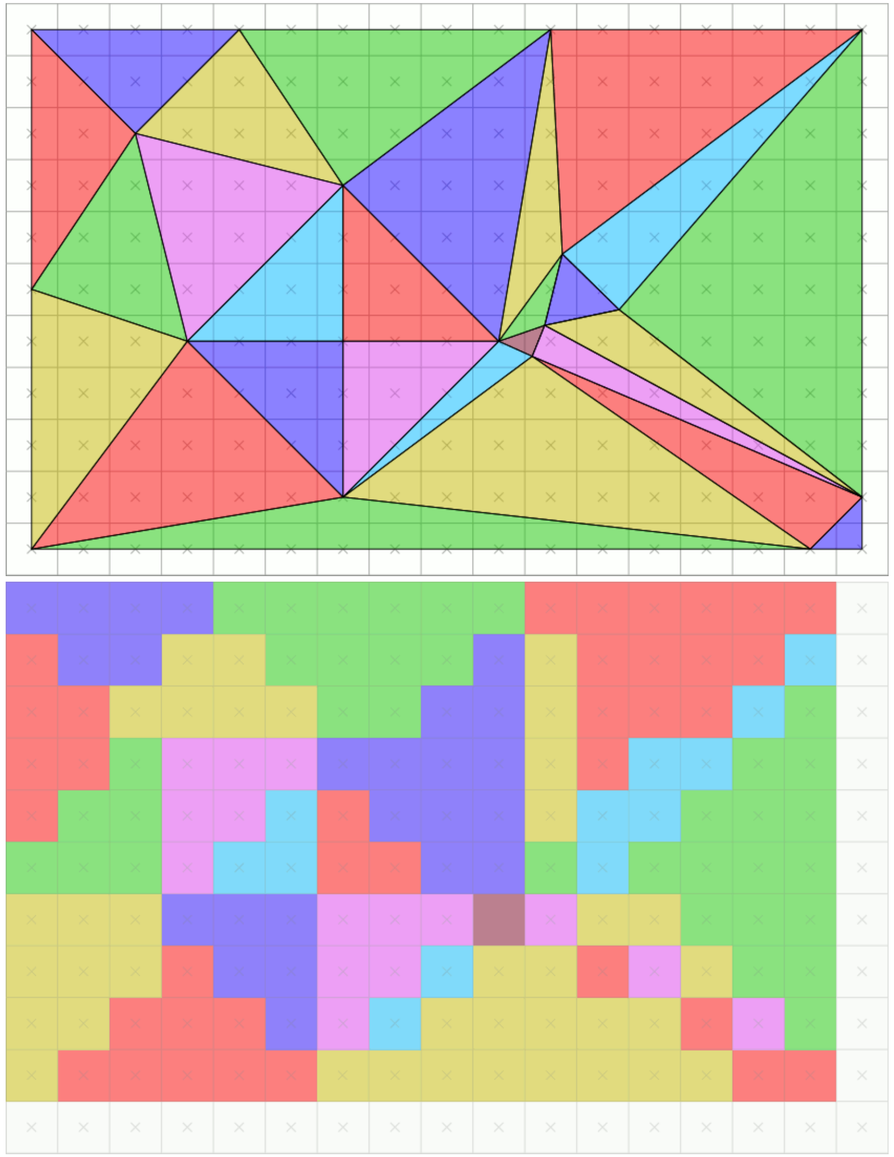
Rasterization

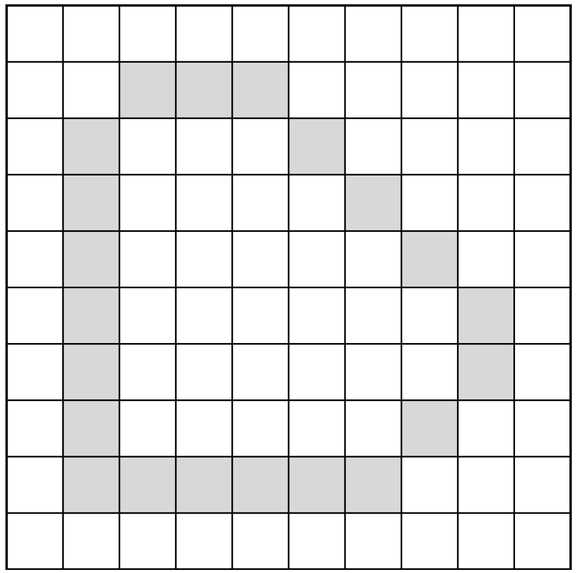


Fragments

- Rasterization is concerned with the formation of fragments of a raster image from vector graphics based geometric primitives
  - discretization of geometry after projection from 3D camera space to 2D screen space
  - a.k.a. **scan-conversion**
- Fragments are potential pixels
  - have a color attribute and location in screen space
  - carry depth information for visibility determination in 3D
    - multiple fragments can be generated for one pixel from all geometric primitives projecting onto the same pixel



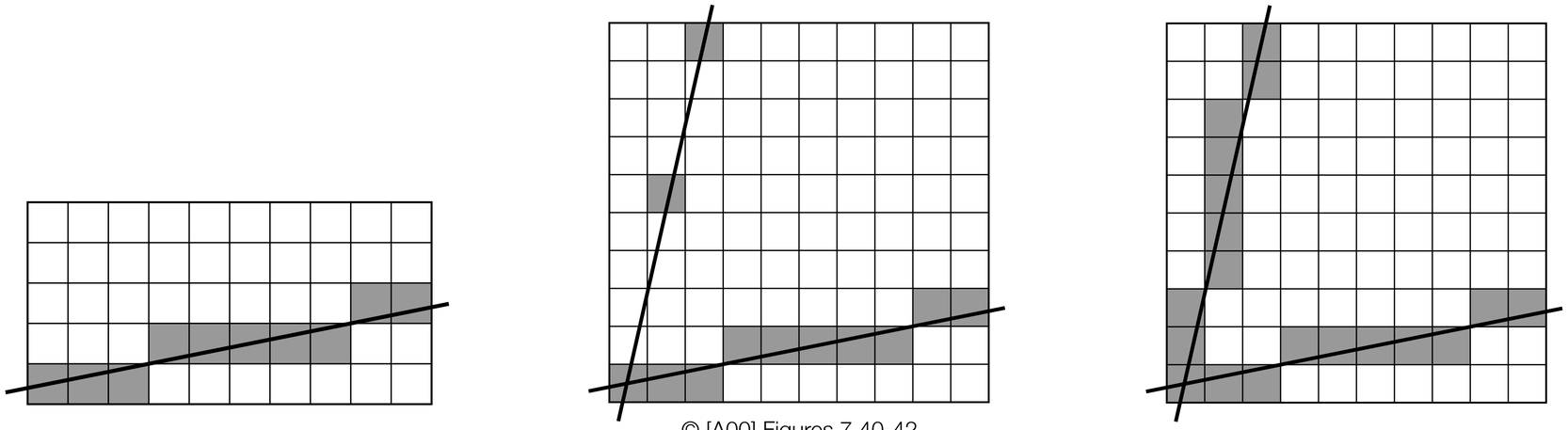
Scan-Conversion



© [A00] Figure 7.52

- Scan-conversion algorithms compute the coordinates of fragments that lie on or near a specified geometric primitive
  - must be **as fast as possible** as many million geometric primitives must be drawn in a small fraction of a second
- Drawing lines or filling polygons on a raster typically causes discretization artifacts, i.e. an aliasing problem
  - antialiasing techniques to remedy such problems

Line Symmetry



© [A00] Figures 7.40-42

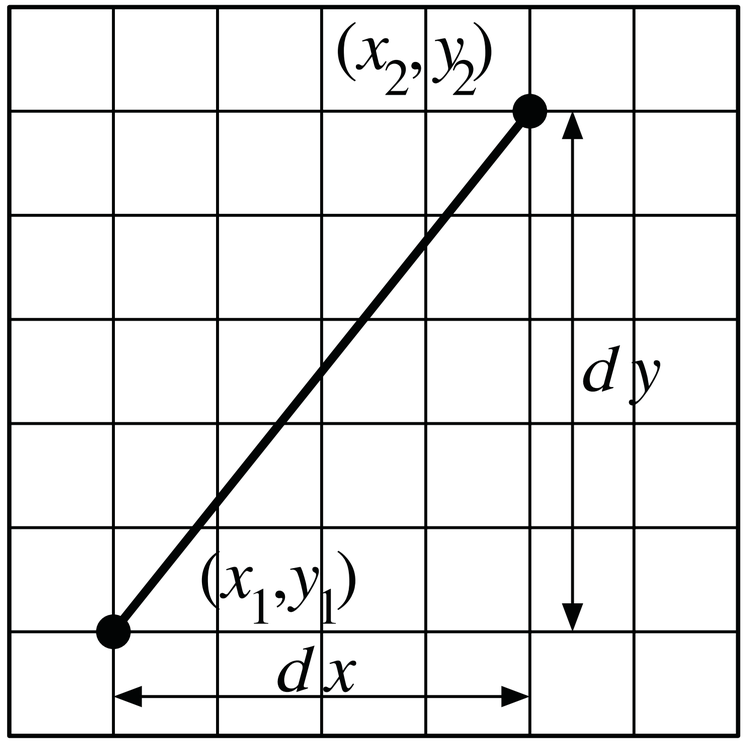
- For slopes m larger than 1.0, pixels in the y direction are skipped since we advance y by more than one pixel at a time
- Can easily be resolved by swapping the roles of x and y for larger slopes
  - adjust also for negative slopes
  - removes potential problems of horizontal or vertical lines $(m{=}\infty)$
- Implement algorithm for standard positive slopes m less than 1.0

Simple Line Drawing

- Assume slope $m = dy/dx$ in $[0,1]$ between two points
  - explicit line equation is $y = y_1 + m \cdot (x - x_1)$
- A unit change $dx = 1.0$ along x causes y to change by $dy = m$ between y_1 and y_2
- Fill pixels increasing in x from x_1 to x_2

```
float y = y1, m = (y2-y1)/(x2-x1);
for (x = x1; x <= x2; x++) {
  WritePixel(x, (int)floor(y+0.5), color);
  y += m;
}
```

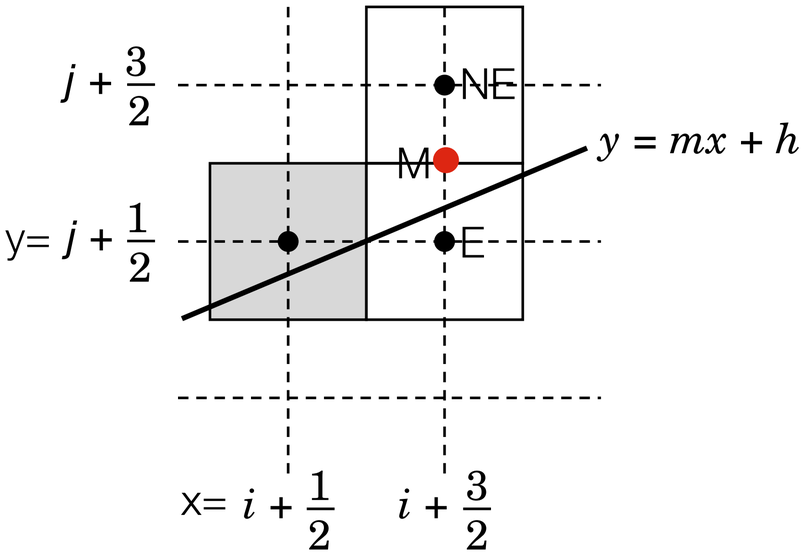
  - since float variables have limited precision, accumulation of m may cause cumulative error buildup for long lines



© [A00] Figure 7.39

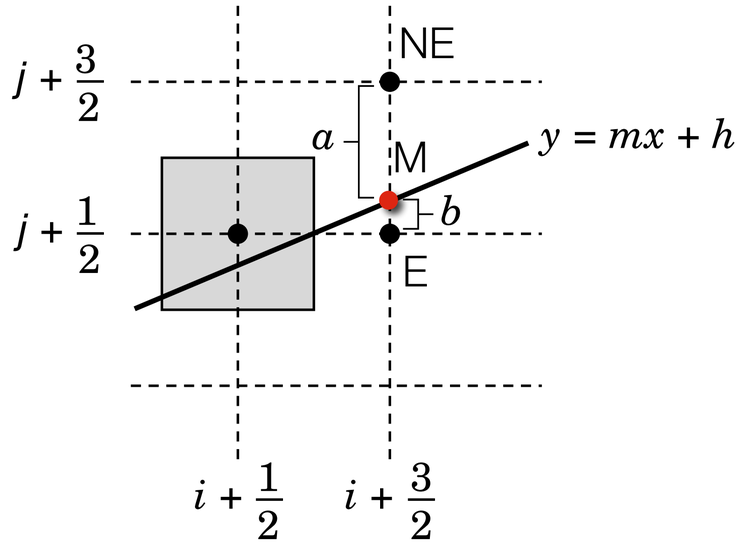
Bresenham's Algorithm

- Floating point and expensive rounding operations are time consuming if done in "gazillions"
  - Bresenham developed a scan-conversion avoiding floating point calculations
- Decide for each step if the next pixel:
  is to the East (E) or Northeast (NE)
  - decision is made based on the midpoint M between the $E = (x+1, y)$ and the $NE = (x+1, y+1)$ pixel
    - with the pixel centers $x = i+1/2$ and $y = j+1/2$
  - if M is above the given line, $M > y_1 + m \cdot ((x+1) - x_1)$ then the next pixel to fill is E, else it is the NE pixel
- All computations and decisions can be made using integers



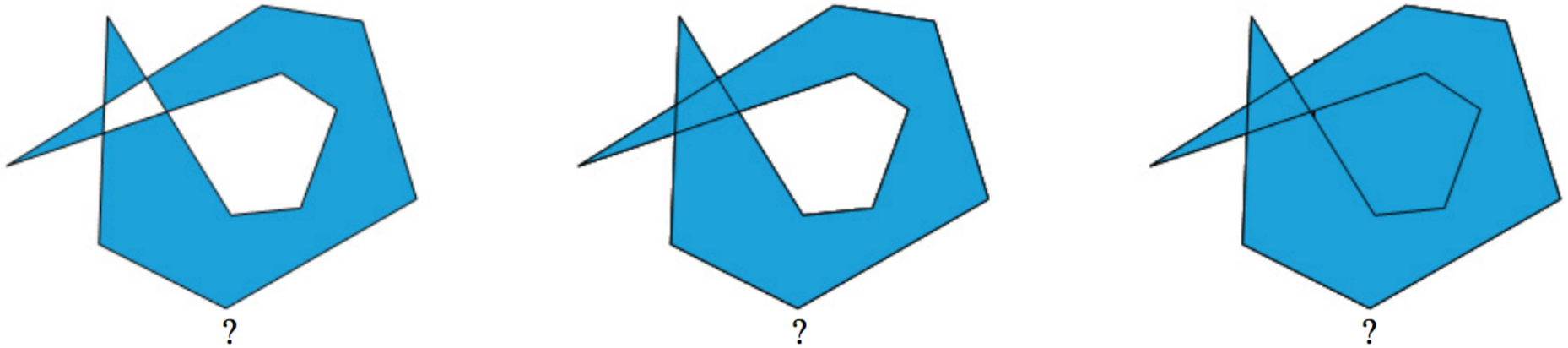
© [A00] Figure 7.43

- Use of decision variable $d = a–b$ with $d > 0$ indicating E or $d < 0$ NE as the next pixel to be filled
  - going E: $a = a - 1 \cdot m = a - dy/dx, \quad b = b + 1 \cdot m = b + dy/dx$
    going NE: $a = a + 1 - m = a + 1 - dy/dx, \quad b = b - 1 + m = b - 1 + dy/dx$
  - but $\pm m = dy/dx$ is a float number, transform decision to only use integers
  - use non-rational decision variable $dx \cdot d = dx \cdot (a–b)$ without affecting the inequality $d > \text{or} < 0$
  - updating $dx \cdot d = dx \cdot a - dx \cdot b$, will use $\pm dy$ instead of $\pm\, dy/dx$
- We can update d incrementally either by $\Delta d_E = -2dy$ or $\Delta d_{NE} = 2(dx–dy)$, for going E or NE
  - going E: $dx \cdot d = dx \cdot a - dy - dx \cdot b - dy = dx \cdot a - dx \cdot b - 2dy = dx \cdot d - 2dy$
  - all integer only calculations to update
- Initialize d for the first step to $dx - 2dy$
  - for the first pixel at x_1 where the line intersects at y_1, set $a = 1$ and $b = 0$
  - first test at x_1+1 then uses $d = dx \cdot (a–b) + \Delta d_E = dx - 2dy$



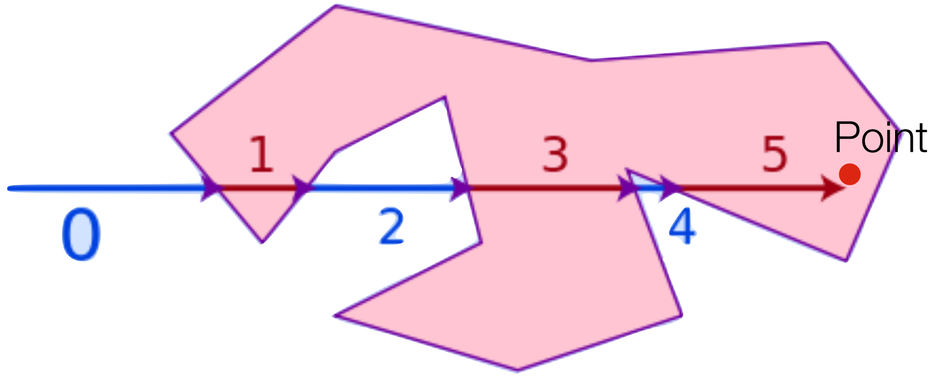
© [A00] Figure 7.44

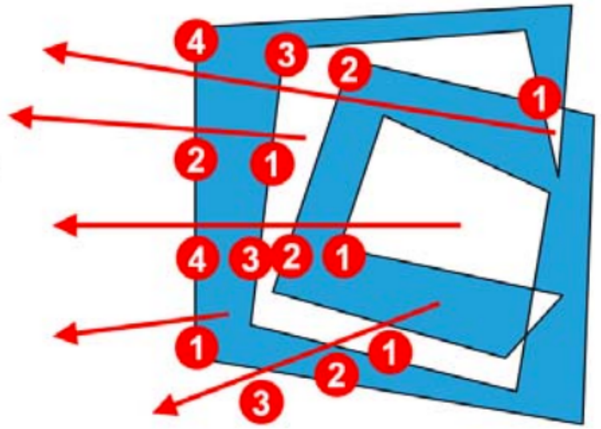
Polygon Filling



- *Main Question*: How to fill the interior of a polygon with a color
  - inside-outside are only well defined for flat simple polygons
- Typically: Fill interior of polygon with color during scan-conversion
  - convexity further simplifies scan-conversion filling process
  - non-convex and self-intersecting polygons complicate the process

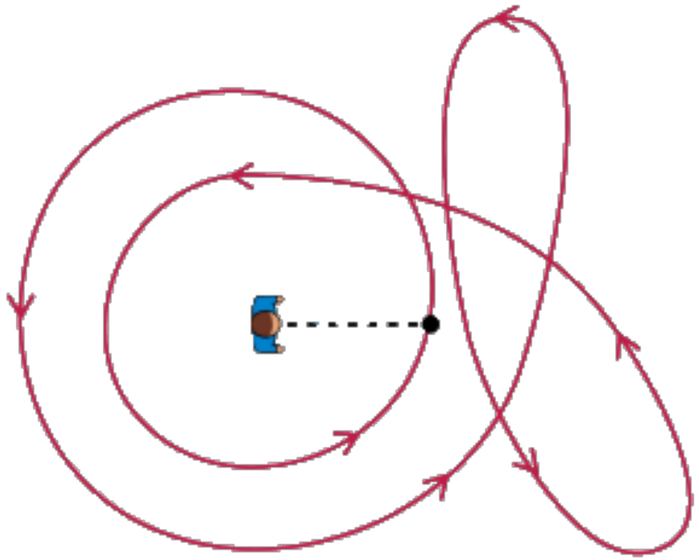
Odd/Even Inside/Outside Test

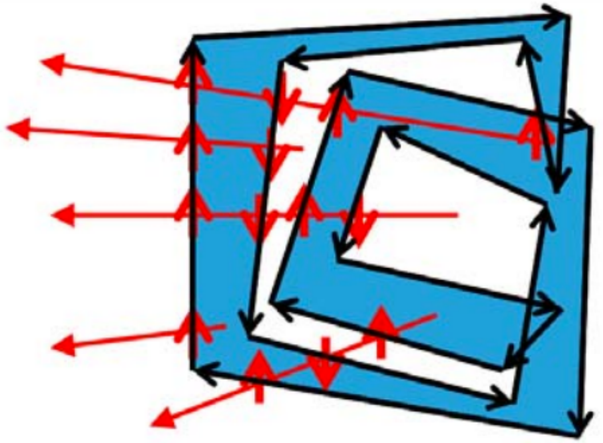




- Odd-even test is simple to implement
  - how many times a ray is crossing a polygon edge from infinity to a given query point
- Related to edge-intersection pairs in scan-line filling
  - test can be integrated with the rasterization of polygons

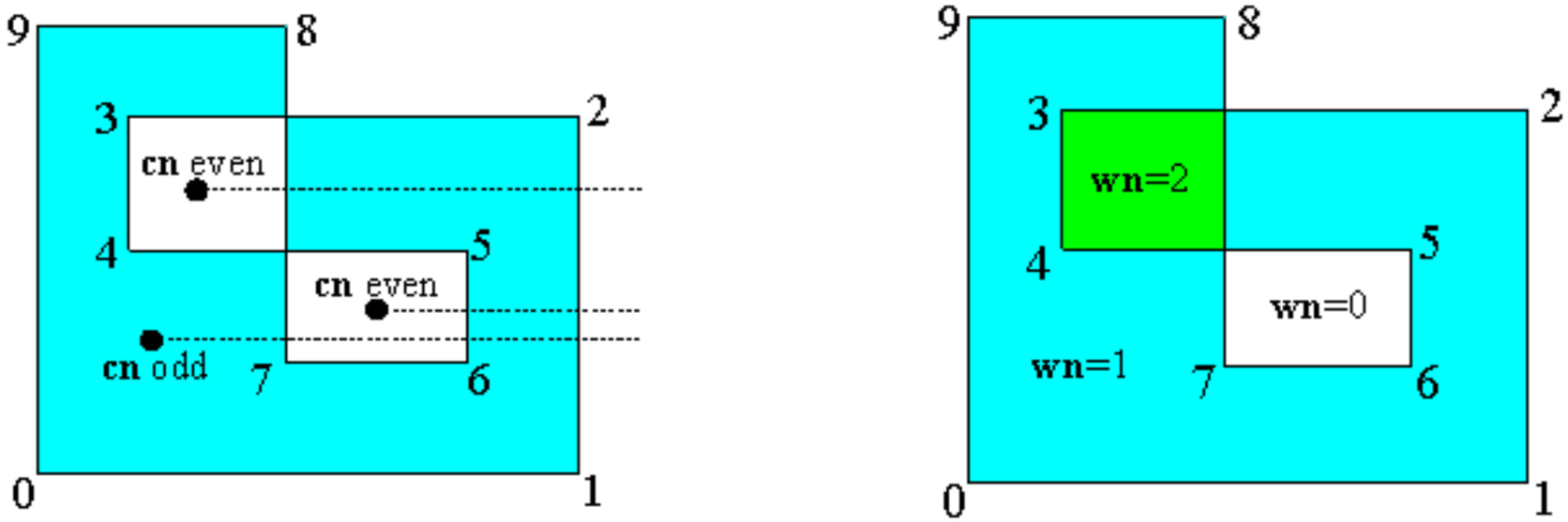
Winding Number Inside/Outside Test





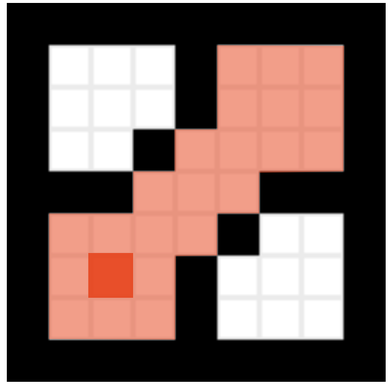
- Nonzero-Winding number rule: times circling around a point
  - point is outside if there is the same number of clockwise and counterclockwise directions
  - if winding number > 0 then point is inside
  - stable also for non-simple, non self-intersecting polygons

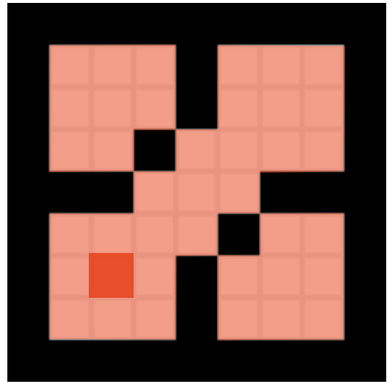
Odd/Even vs. Winding Number



Source http://geomalgorithms.com/a03-_inclusion.html

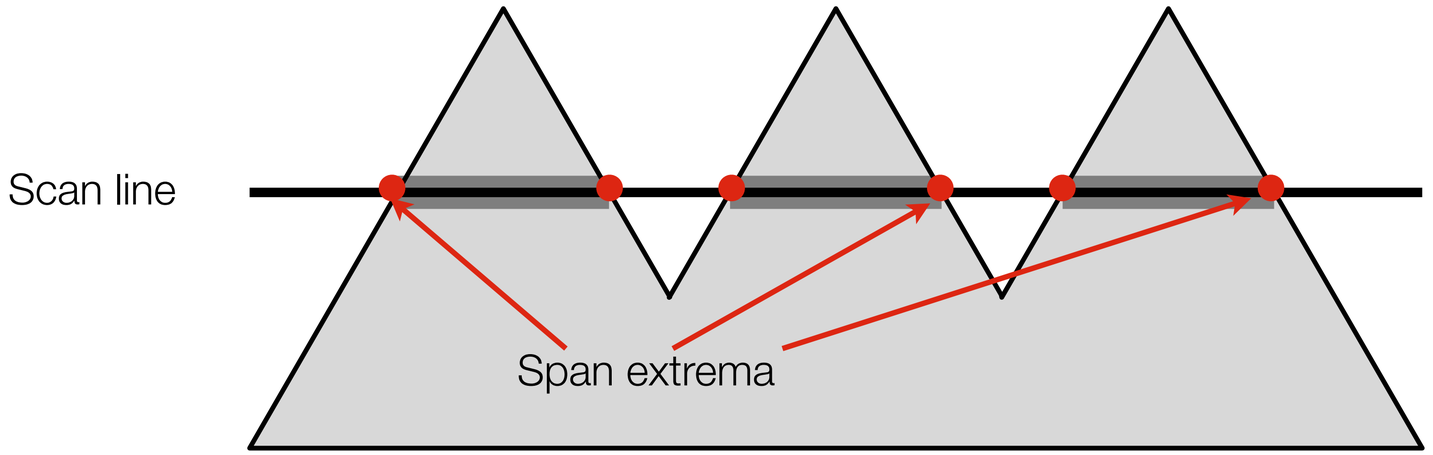
Pixel based Flood Filling





- Given an interior seed point, neighboring pixels can recursively be colored until reaching any prior rasterized polygon edge
  - recursion and duplicate work can be avoided using clever ordering and queuing strategies
- Different 4- and 8-way neighborhood filling results

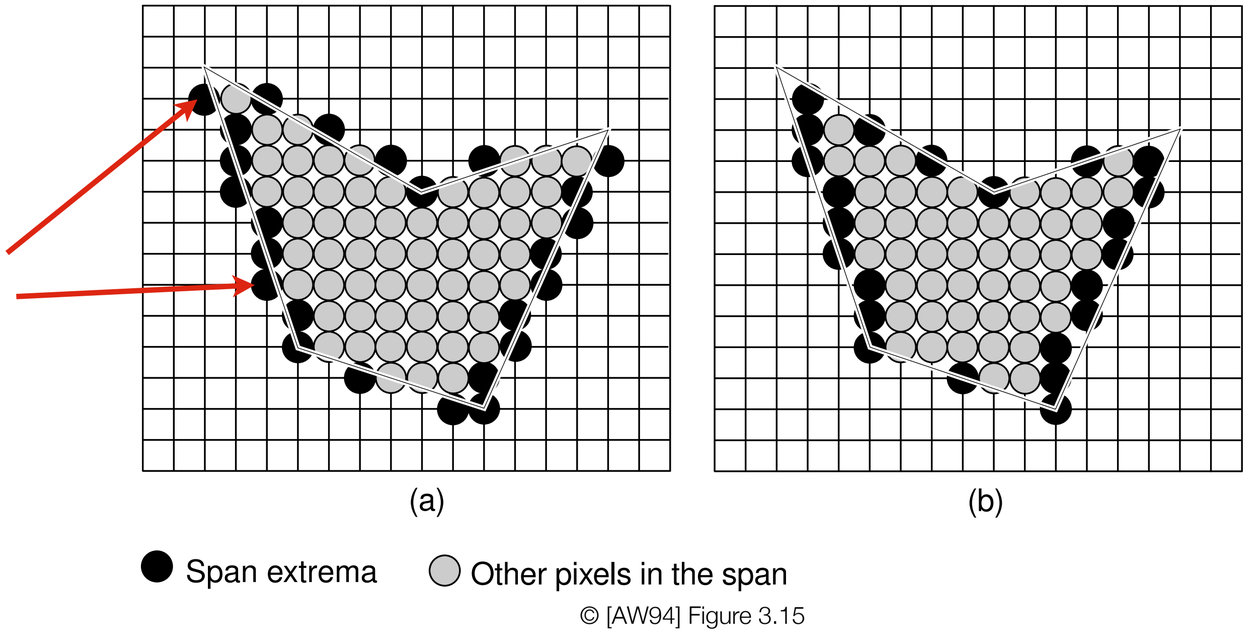
Scan-Line Filling



© [A00] Figure 7.53

- Fill **spans** of pixels on a scan line between the left and right edges of a polygon
  - works for convex as well as concave polygons
  - similar to odd-even test
- Span extrema required for each scan line during rasterization
  - maintained during polygon edge scan-conversion

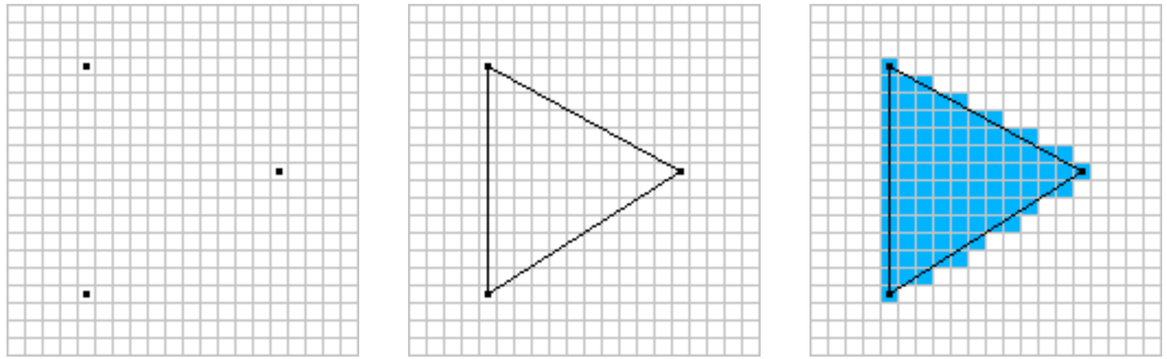
Polygon Boundaries



© [AW94] Figure 3.15

- Scan-conversion of polygon edges in general may produce pixels not necessarily inside the polygon
  - ambiguous coloring for adjacent polygons sharing the same edge
  - modified polygon edge scan-conversion algorithm required
    - pixels must be on the inside side of the polygon edge

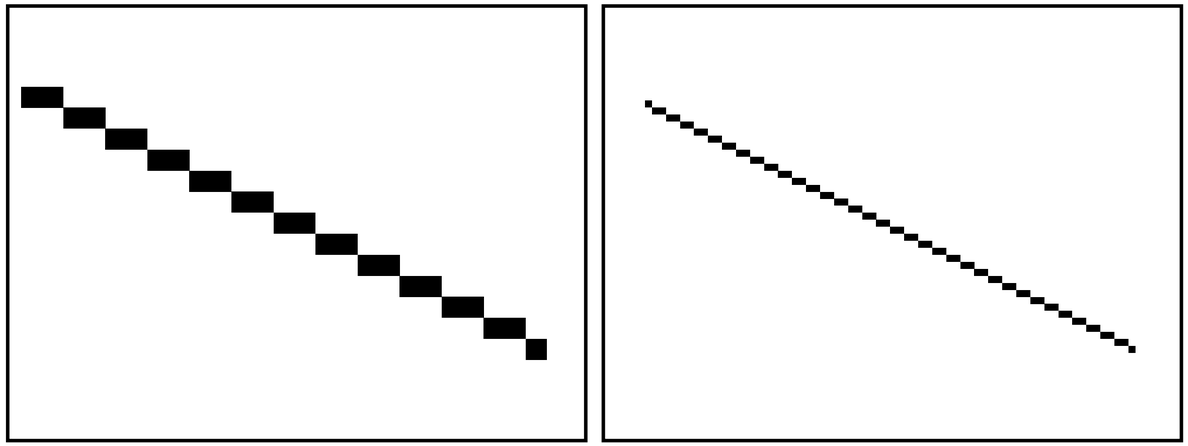
Triangle Rasterization



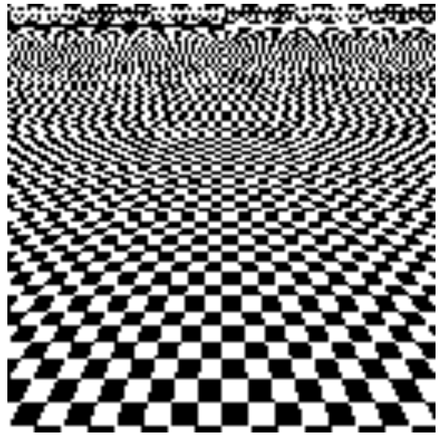
© 2022 Josh Beam, Source: https://joshbeam.com/articles/triangle_rasterization/

- Triangles are the preferred polygons to represent 3D shapes
  - guaranteed convex and planar polygons
- Rasterization of polygons can be achieved through rasterizing triangles
  - individual polygons broken up into triangles
- Exploit optimized triangle rasterization algorithm
  - determine the two edges of the triangle intersecting the scanline
  - loop through the y axis range of the triangle to calculate the horizontal spans
  - fill pixels between span endpoints
- Care has to be taken to not leave holes between adjacent triangles and to not rasterize any pixel more than once

Antialiasing



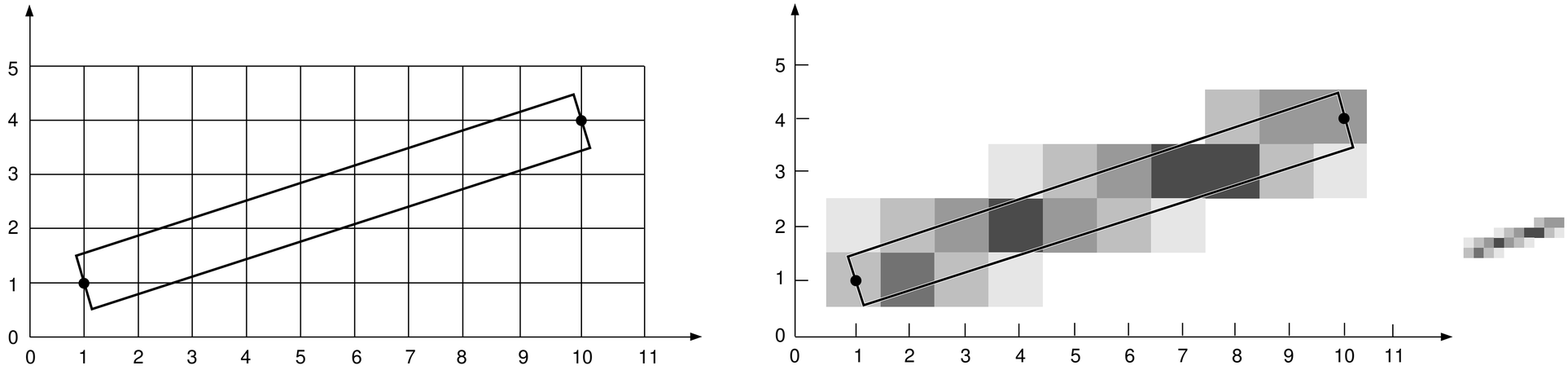
© [A00] Figure 7.59



- Aliasing problems are artifacts caused from discretization and sampling of a continuous signal
  - jagged staircase lines, moiré patterns
- Key is to remove signal components with higher frequency than sampling resolution
  - increasing the resolution does not eliminate the problem
  - variable pixel intensities can smooth out abrupt intensity changes

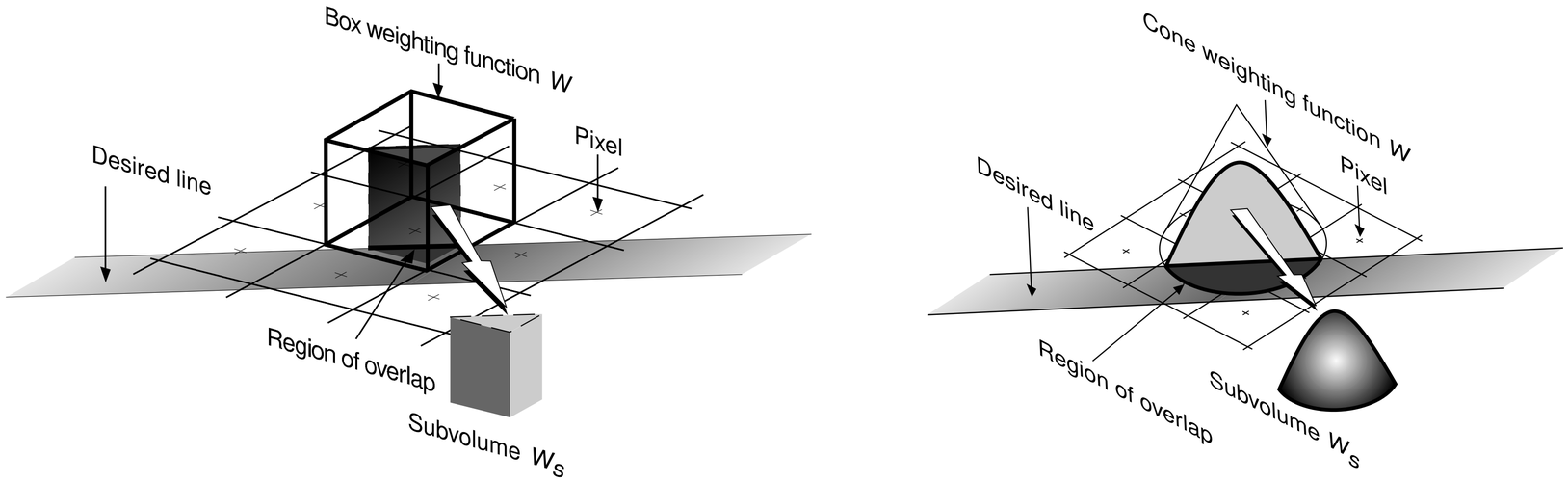
Line Width and Area

- Line has arbitrary width and occupies finite area on screen
  - discrete B/W pixels cannot approximate line area well
- Amount of distributed color (ink) per pixel can be adjusted to match intensity and area of line
  - using varying per-pixel color intensity depending on how much is covered by the line
  - smaller than pixel-width lines possible also possible



© [AW94] Figure 3.35,36

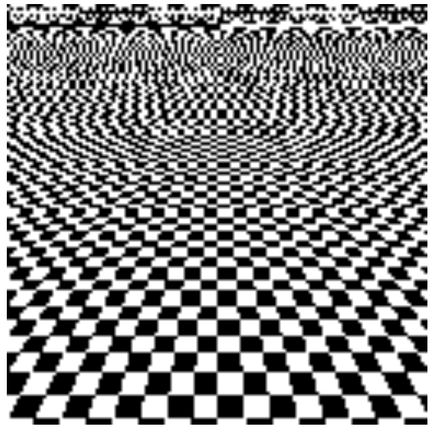
Area Sampling

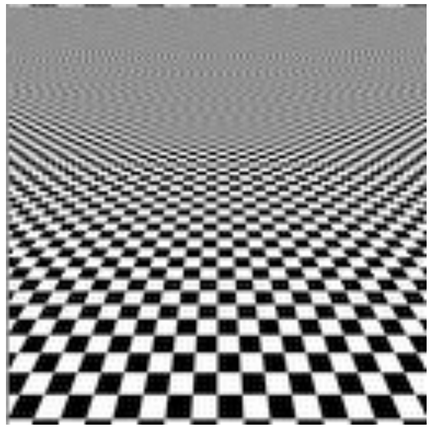


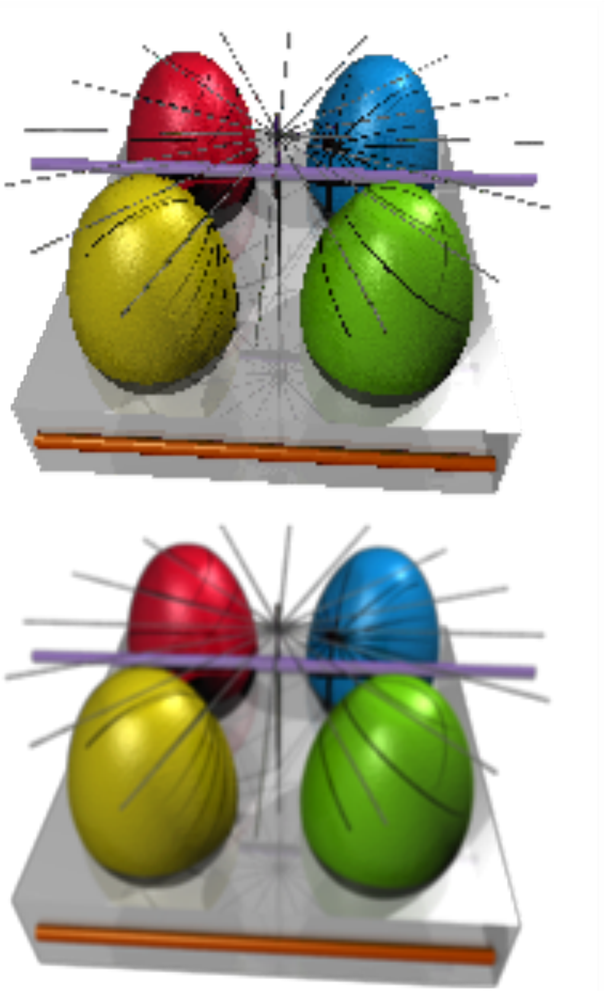
© [AW94] Figure 3.38,39

- Compute the partial area of a pixel covered by the line and color it proportionally to that
  - **unweighted**: intensity is equal to the % of the pixel's intersection area with the line
    - same intensity for equal area overlap
  - **weighted**: intensity is equal to the integral over the weighting kernel's intersection region with the line
    - overlap area closer to pixel center causes higher intensity

Area Antialiasing







- Texture antialiasing filters
  - multiple checkerboard squares contribute to the topmost pixels
- FSAA attempts to average the contribution of multiple color contributions per pixel
  - supersampling usually means that each frame is rendered at double (2x) or quadruple (4x) resolution
  - finally down-sampled to match the display resolution, 2x = four supersampled pixels

Copyrights

- Many figures in these slides are copyright protected as indicated. Text and all other figures are copyright protected by Renato Pajarola.

- [AW94] Electronic Instructors Manual (EIM) for Introduction to Computer Graphics by James Foley, Andries van Dam, Steven Feiner, John Hughes, and Richard Phillips. Copyright Addison Wesley 1994.

- [A00] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2000. (http://www.cs.unm.edu/~angel/BOOK/SECOND_EDITION/)

- [AC97] Learning Rendering CD accompanying 3D Computer Graphics by Alan Watt. Copyright A. Watt and L. Cooper 1997. (Permission for copying and distribution without direct commercial advantage. To copy otherwise, or to republish, requires specific permission.)

- [S02] Electronic Figures for Fundamentals of Computer Graphics by Peter Shirley. Copyright P. Shirley 2002. (http://www.cs.utah.edu/~shirley/fcg/figures/)